

Advanced Software Development

Johan Bontes (johan@digitsolutions.nl)

CSC3003S

31 JULY 2023

ASD 02
Software engineering methods

A. Faster, better, cheaper (pick two)

B. Problems

C. Naïve approaches

D. Iteration

Observations

- Most common problem in software systems is not the construction, but the *estimation*.
- Software projects **fail to meet cost and schedule,** because those targets are wrong.
  - Costing software = difficult
- Know little about accurate estimations so targets are *unreasonable*
  - made by people least able to make them
    - e.g., marketers, managers and customers.
- Communicating abstract or conceptual ideas is hard.



Software engineering triangle

- 1990s, NASA adopted the slogan:

  “Faster, better, cheaper.”

- Following a series of unsuccessful projects they abandoned the slogan.

  “Faster, cheaper, better”

  is impossible to satisfy in a project.

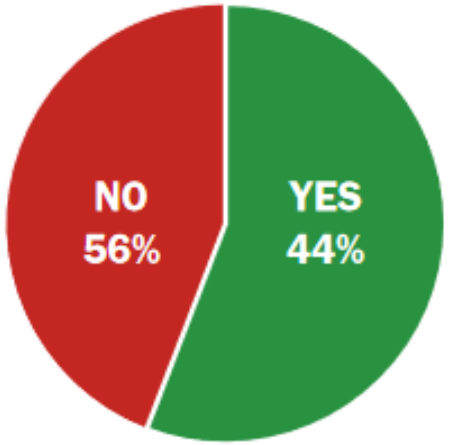
- Software engineers say that a correct statement of NASA's praiseworthy goal is

  “Faster, better, cheaper: pick 2 out of 3”



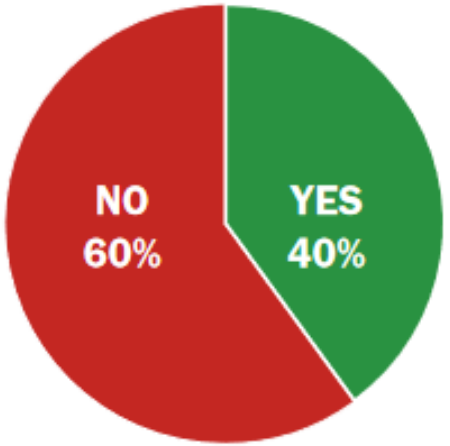
Standish Chaos Report 2015

ONBUDGET



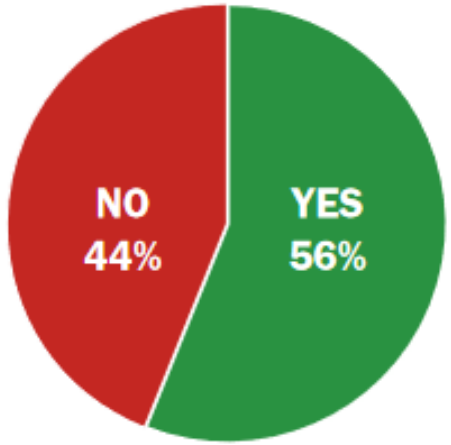
The percentage of projects that were OnBudget from FY2011–2015 within the new CHAOS database.

ONTIME



The percentage of projects that were OnTime from FY2011–2015 within the new CHAOS database.

ONTARGET



The percentage of projects that were OnTarget from FY2011–2015 within the new CHAOS database.

TRADITIONAL RESOLUTION FOR ALL PROJECTS

	2011	2012	2013	2014	2015
SUCCESSFUL	39%	37%	41%	36%	36%
CHALLENGED	39%	46%	40%	47%	45%
FAILED	22%	17%	19%	17%	19%

Resolution by large and small projects

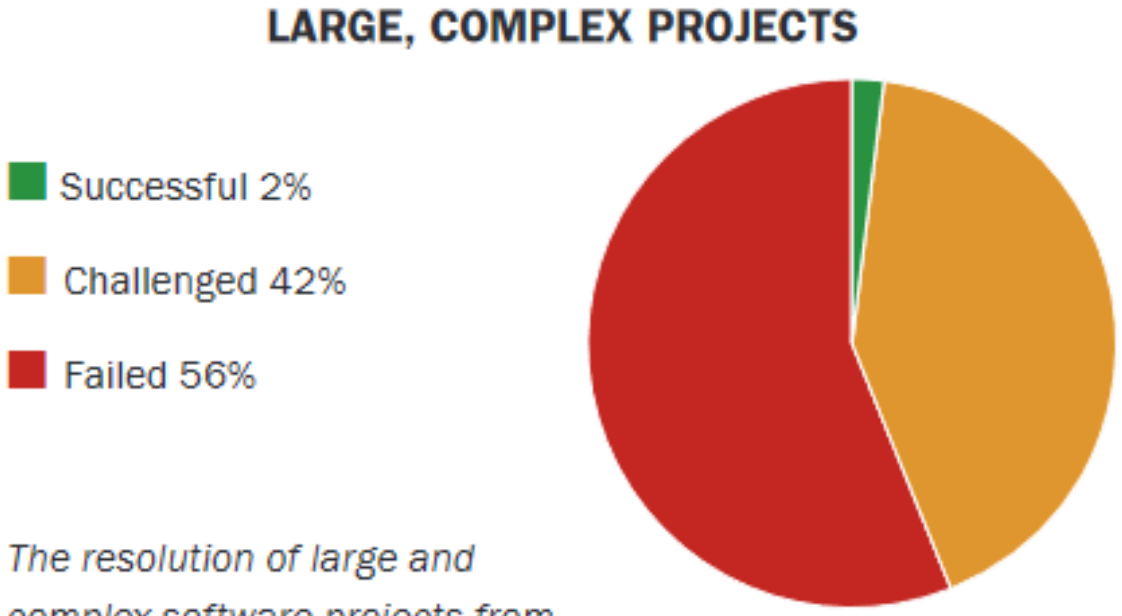
PROJECT SIZE BY CHAOS RESOLUTION

	SUCCESSFUL	CHALLENGED	FAILED	TOTAL
Grand	6%	51%	43%	100%
Large	11%	59%	30%	100%
Medium	12%	62%	26%	100%
Moderate	24%	64%	12%	100%
Small	61%	32%	7%	100%

The size of software projects by the Modern Resolution definition from FY2011–2015 within the new CHAOS database.

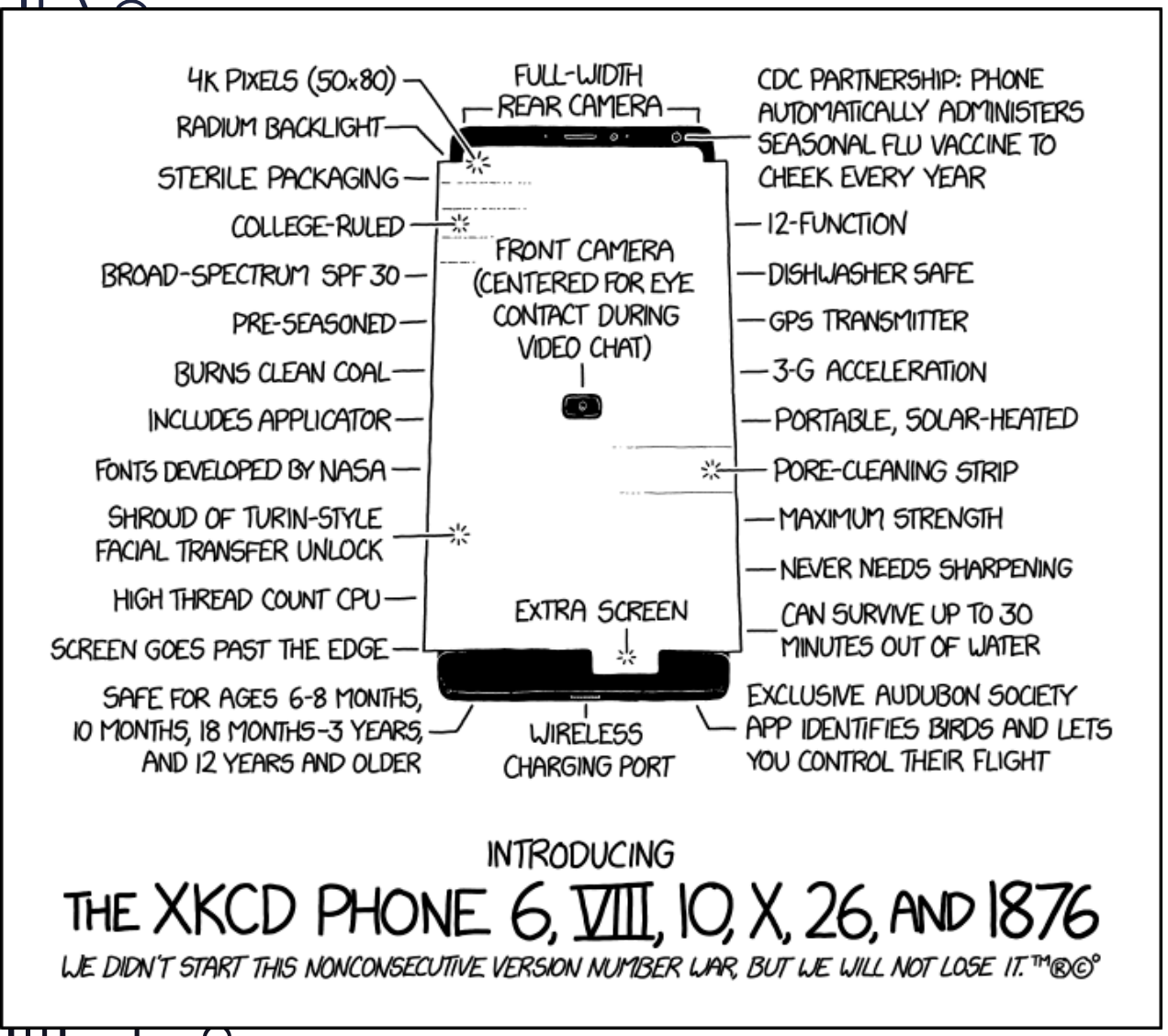
"Think Big, Act Small": Just say no (to large projects)

- Secret to project success: *limit* size and complexity
  - Size + complexity > all other success factors.
- *Break large projects down* into a sequence of smaller ones, prioritized on direct business value.
  - Use stable, full-time, cross-functional teams that follow a disciplined approach.
- Quick solution: just say no to large projects
  - more sensibly: adopt a small project strategy.
  - lower cost, with fewer defects.
- Projects too often get too big to succeed.
  - Constantly being called on to do more for less
  - But the real key to success is doing *less* for less



The resolution of large and complex software projects from FY2011–2015 within the new CHAOS database.

CHAOS Resolution by Agile vs Waterfall				
Size	method	success	challenge	fail
All size projects	Agile	39%	52%	9%
	Waterfall	11%	60%	29%



ASD02
Software engineering methods

A. Faster, Better, Cheaper (Pick Two)

B. Problems

C. Naïve Approaches

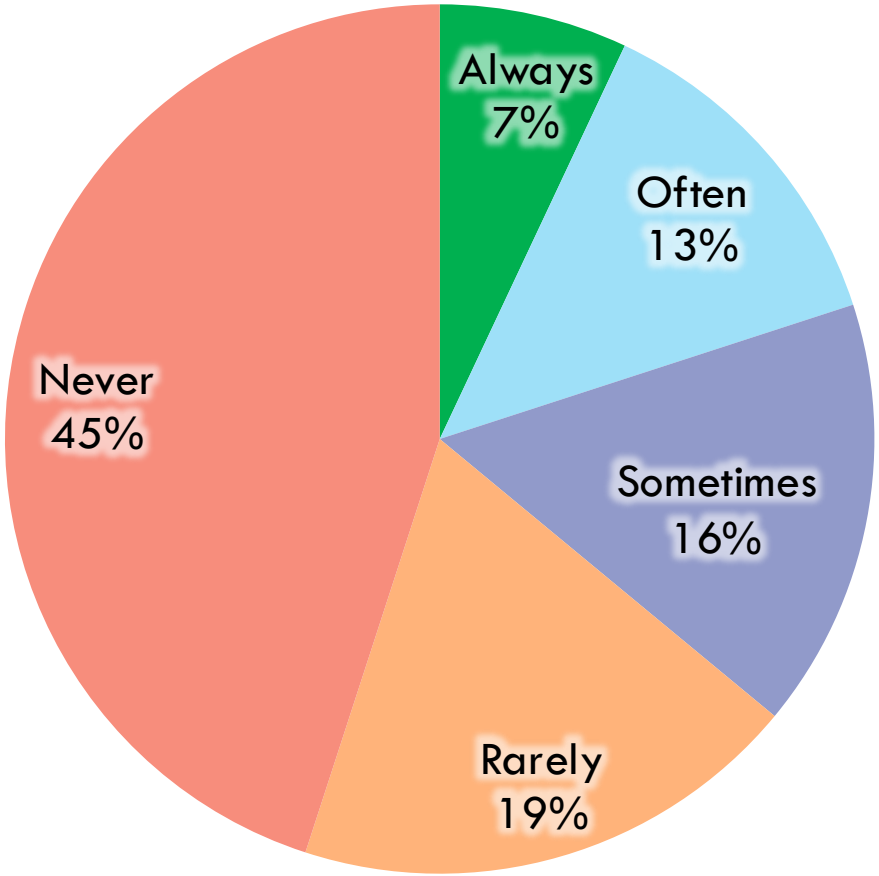
D. Iteration

Wasted effort

45% of features are never used, while another 19% are used rarely

➔ Almost 2/3 of the features are never or rarely used!

➔ Stop developing these features and double productivity!



Note: there has been increasing criticism of Standish methods. See Eveleens and Verhoef, "The Rise and Fall of the Chaos Report Figures" IEEE Software, **27**, 1, 30–36, Jan.-Feb. 2010, Doi: 10.1109/MS.2009.154 or www.cs.vu.nl/~x/chaos/chaos.pdf

"… the Standish figures for individual organizations don't reflect reality and are highly influenced by forecasting biases. Because the underlying data has an unknown bias, any aggregation of that data is unreliable and meaningless."

YAGNI: You Ain't Gonna Need It

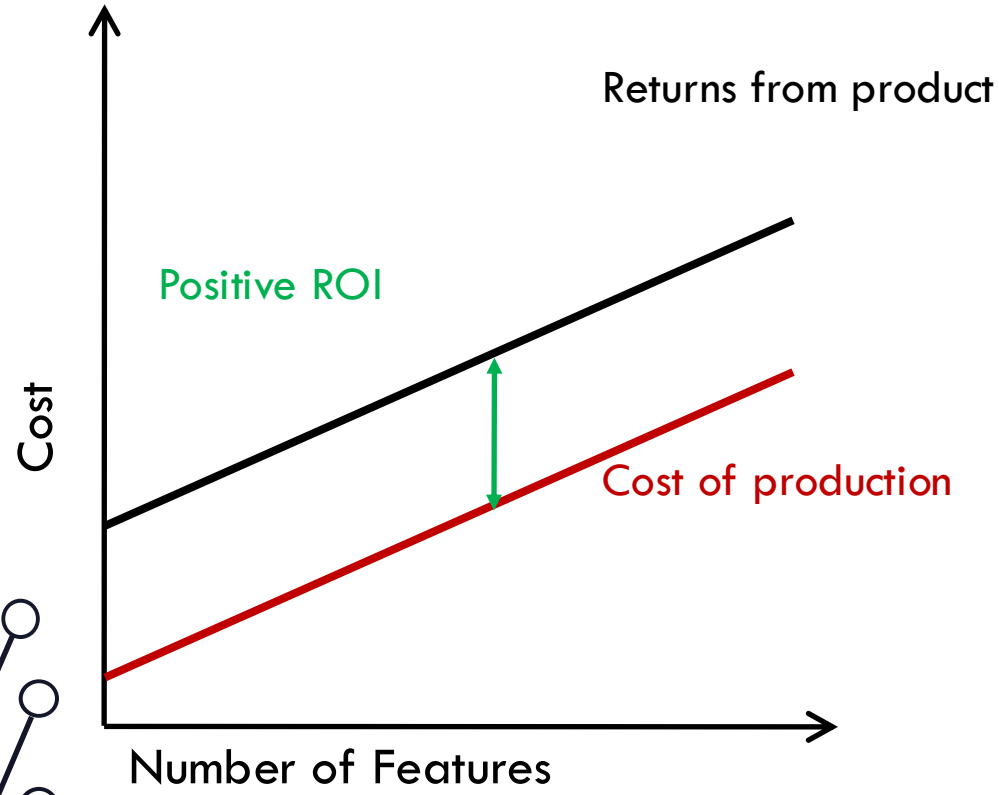
- Try to prevent speculative development and Gold Plating (aka Bells and Whistles).
  - "I'm sure I'm going to need some additional functionality later, so I'll write it now."
- Better is to build only what you need now.
- Speculative development adds complexity to code prematurely.



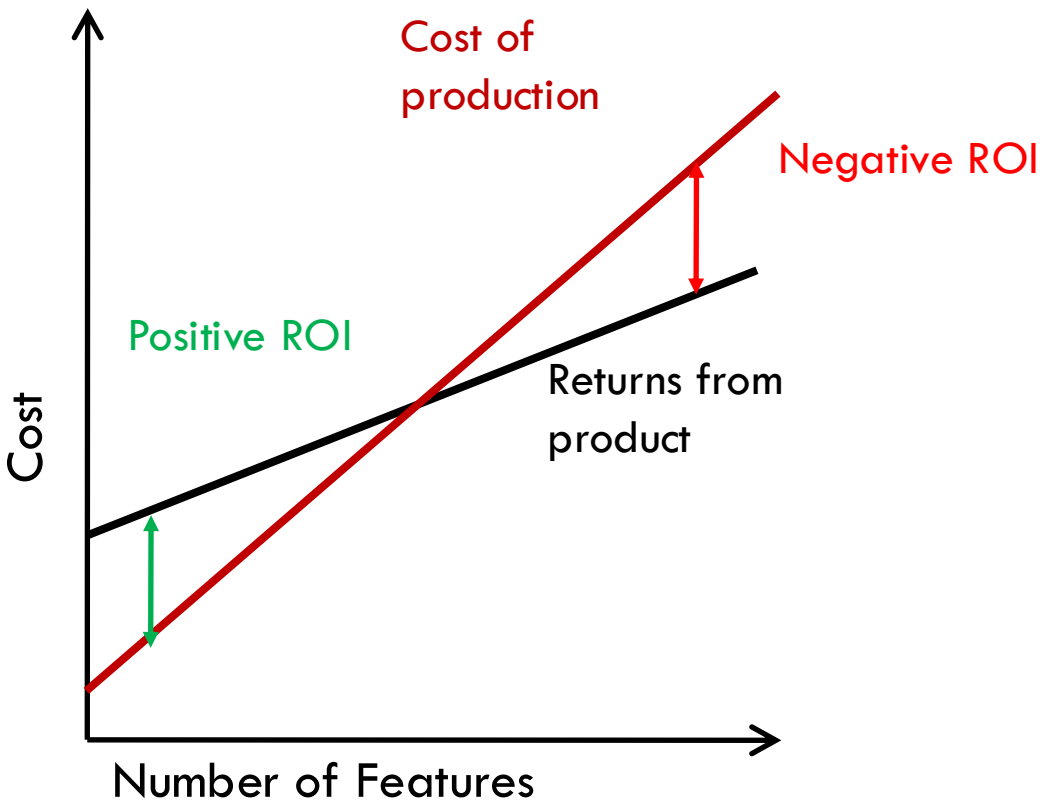
Project Manager: We're going to build a facial-recognition application.
Programmer: OK
Project Manager: People will put in a photo, and it will search a database for matching faces.
Programmer: OK
Project Manager: And while it's doing that, it should flash up, on screen, a slide show featuring *hundreds of other random faces which definitely* ***aren't*** *a match*
Programmer: No that's stupid we're not doing that.

Economies of adding features

Profitable project:
returns outpace costs



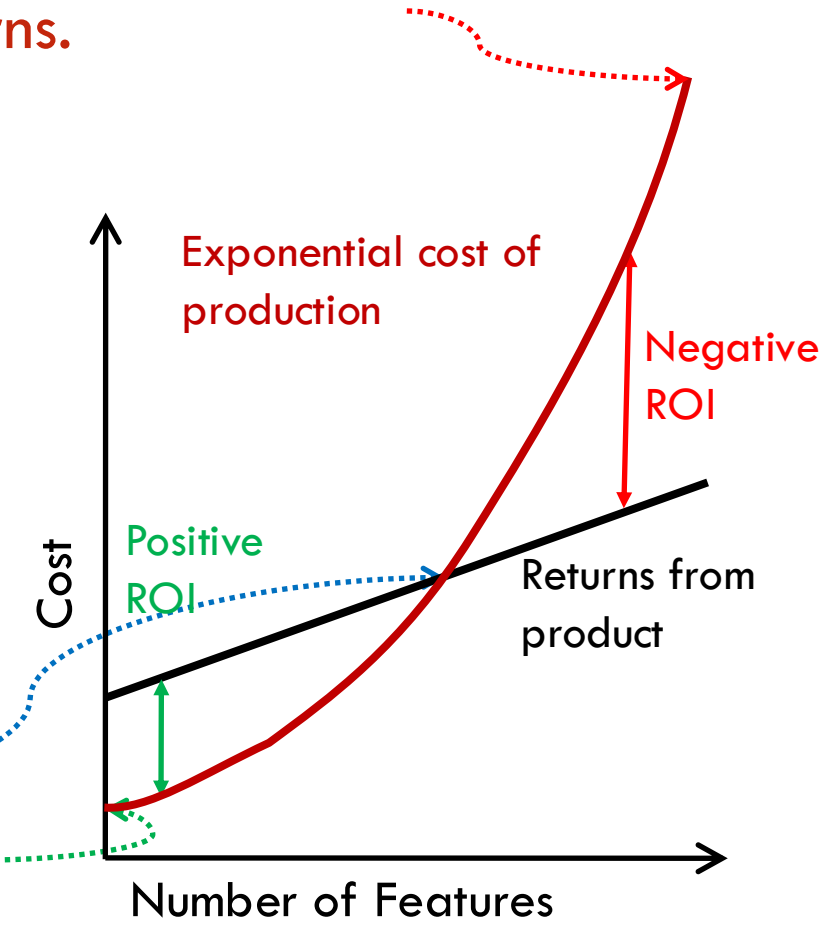
Ultimately unprofitable project:
features become a drag



Cost curve of software is exponential

- Fred Brooks attributes the exponential rise in costs to the cost of communication.
  - E.g., Customer and developer must understand each other perfectly.
- New projects:
  - successful because the cost curve still flat.
- Costs start increasing:
  - quickly overcomes any additional value added from the new features.

Exponential cost overtakes all early returns.



Note: not only new features, also maintenance costs

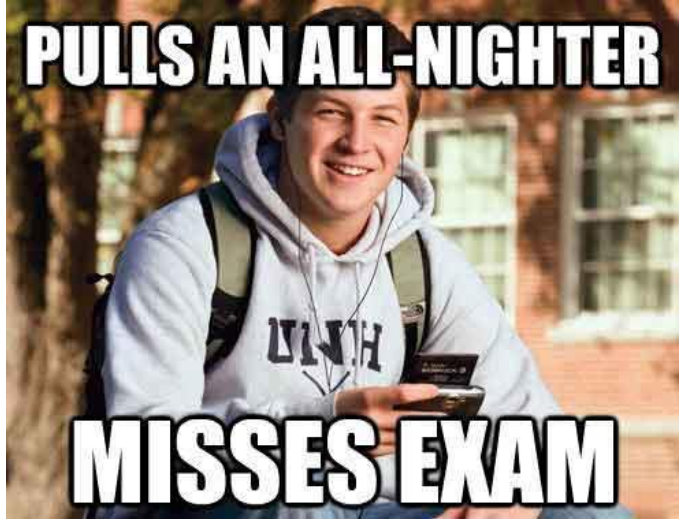
Problems: bad communication



AGE OF BOOTY: (Certain Affinity, Max Hoberman)

- "Over the course of the project there were numerous disconnects between the perceived state of the game and the actual state of the game.
- "The hardest hit were the designers, who continued fine-tuning plans for sophisticated features like matchmaking and party support long after the programmers had already made huge simplifications (and often cuts) to these systems. A combination of lack of attention to the project, poor communication, and wishful thinking led to the design team believing that several features were far more advanced than we were actually able to implement, and they did not find out the reality until very late in the project."

Problems: the crunch



Crunch is the side effect of other problems and the cause of burnout.

Drawn to Life (5th Cell, Joseph Tringali)

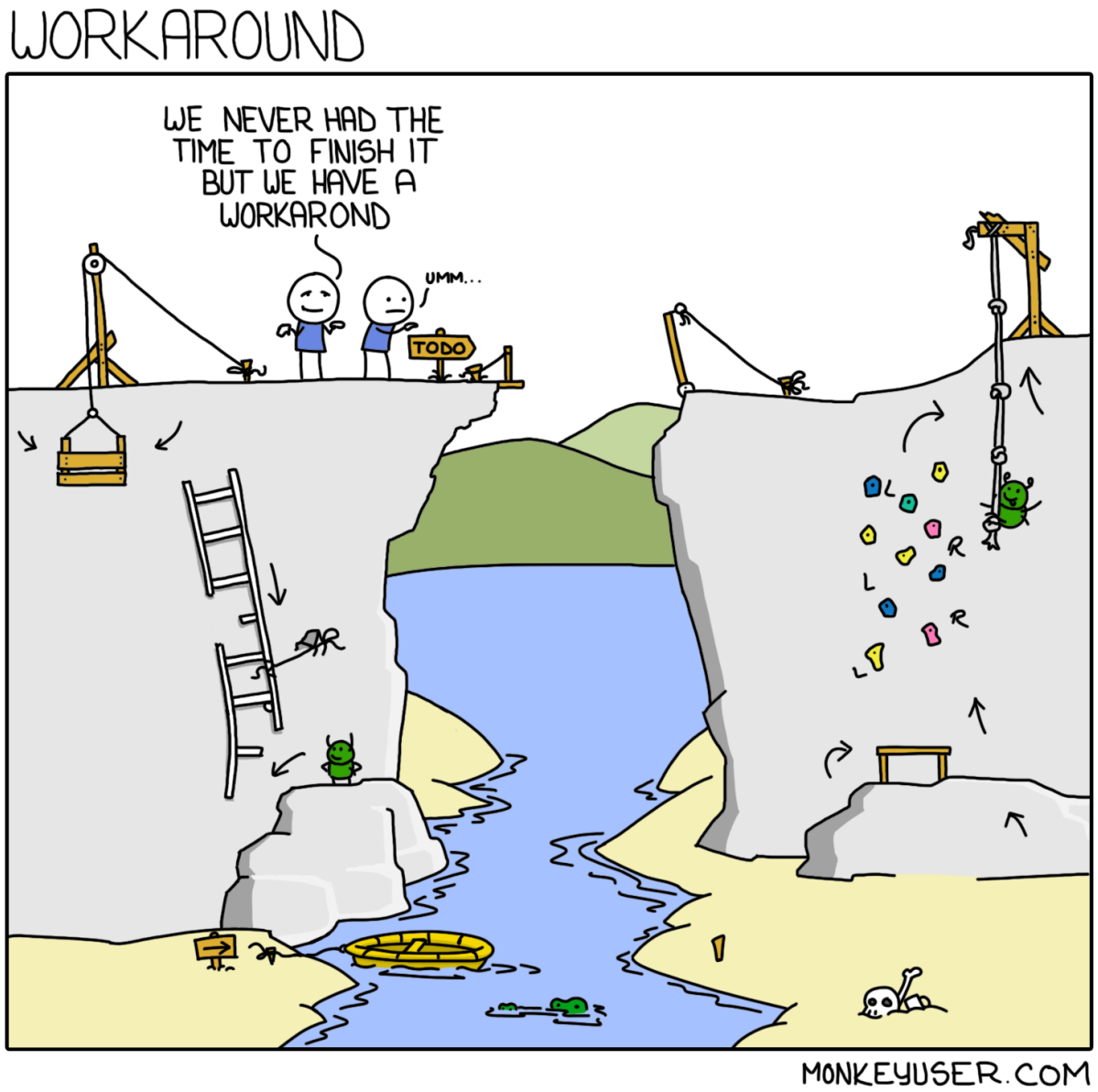
> "Consistent overtime is not fun, nor is it something that should be assumed when developing a game. We ended up working late weekdays and Saturdays for the entire second half of the project.

"Our studio had the perfect storm of factors that contribute to crunch—lack of experience on the platform, an ambitious game, and a schedule that was too short. Combined with so little pre-production, we were playing catch up from day one, taking far too many shortcuts to implement the required game features."



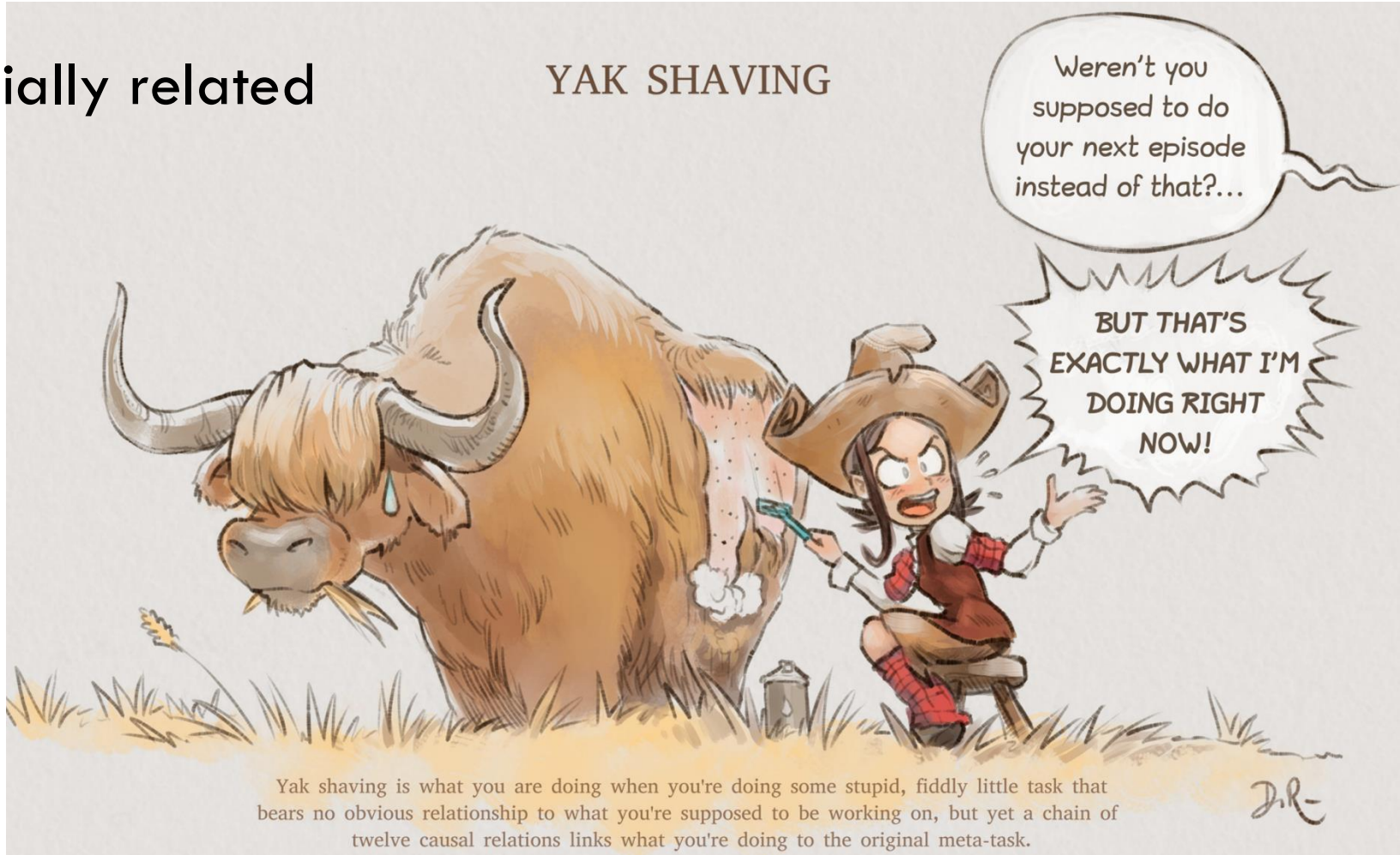
Software Entropy, Rot & Geriatrics

- Entropy = a measure of disorder in a physical system
- Software Entropy: measure of code complexity
  - Tends to increase over time.
  - Overscoped design adds complexity at the start
  - Bug fixes and enhancements increase complexity and degrade structure
    - technical debt
    - most software applications grow at annual rates of 5%–10%
- Entropy makes it hard to
  - make changes and fixes
  - understand the code
- Cure for Entropy is:
  - **YAGNI** at the start and
  - **refactoring** as you go along.



Human factors:

- Yak shaving:
  pointless work tangentially related
- Procrastination:
  (*student syndrome*)
- Gold plating
- Task switching



https://www.youtube.com/watch?v=trXjZ70ab1Q&pp=ygULeWFrIHNoYXZpbmc%3D

Laziness: we'll solve it later

- Essential problems should be spec'ed in early phases
  - Otherwise, you'll do req. work later

    + additional tear down + repair

- Accidental problems should be flagged and analysed when found
  - Split into
    - soln known => refactor
    - soln unknown => back to req.

Methodologies

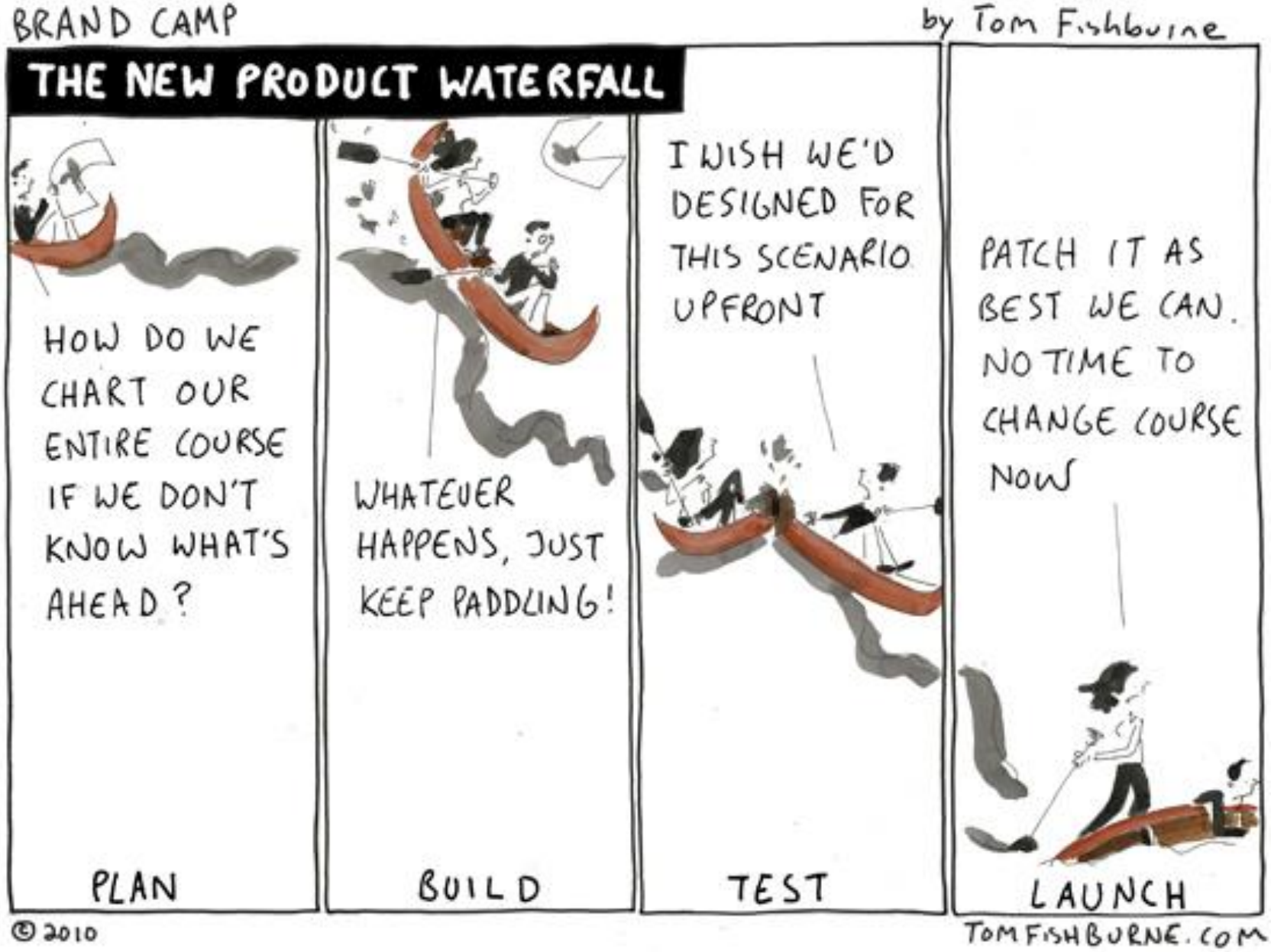
Agile = 1 big project => many small iterative projects

- Risk: too little focus on preparation
  - effort wasted due to late fixes: 100x more expensive
- Fix: Take time to figure out basics
  - e.g.: throw-away prototypes to shake out wicked problems (slide 26)

Waterfall = 1 big project "*eat the whole elephant*"

- Use only if problem well understood (few accidental problems)
- Risk: prep phases (req./analysis) must be near perfect
- Fix: Use all the learnings from previous projects
  Prototype those things that are not fully worked out

ASD Software engineering methods



A. Faster, Better, Cheaper (Pick Two)

B. Problems

C. Naïve Approaches

D. Iteration

Code and fix?

- Naïve, first approach
  - Actually, lack of a methodology
- Little (zero) planning,
  dive straight into implementation
- Reactive
- End with bugs
  - If bugs multiply too fast to fix:
    "death spiral" → cancelled
  - Non-fix: "crunch time"
    - May view as badge of honour,
      but results in burnout



Waterfall method diagram as used in building projects

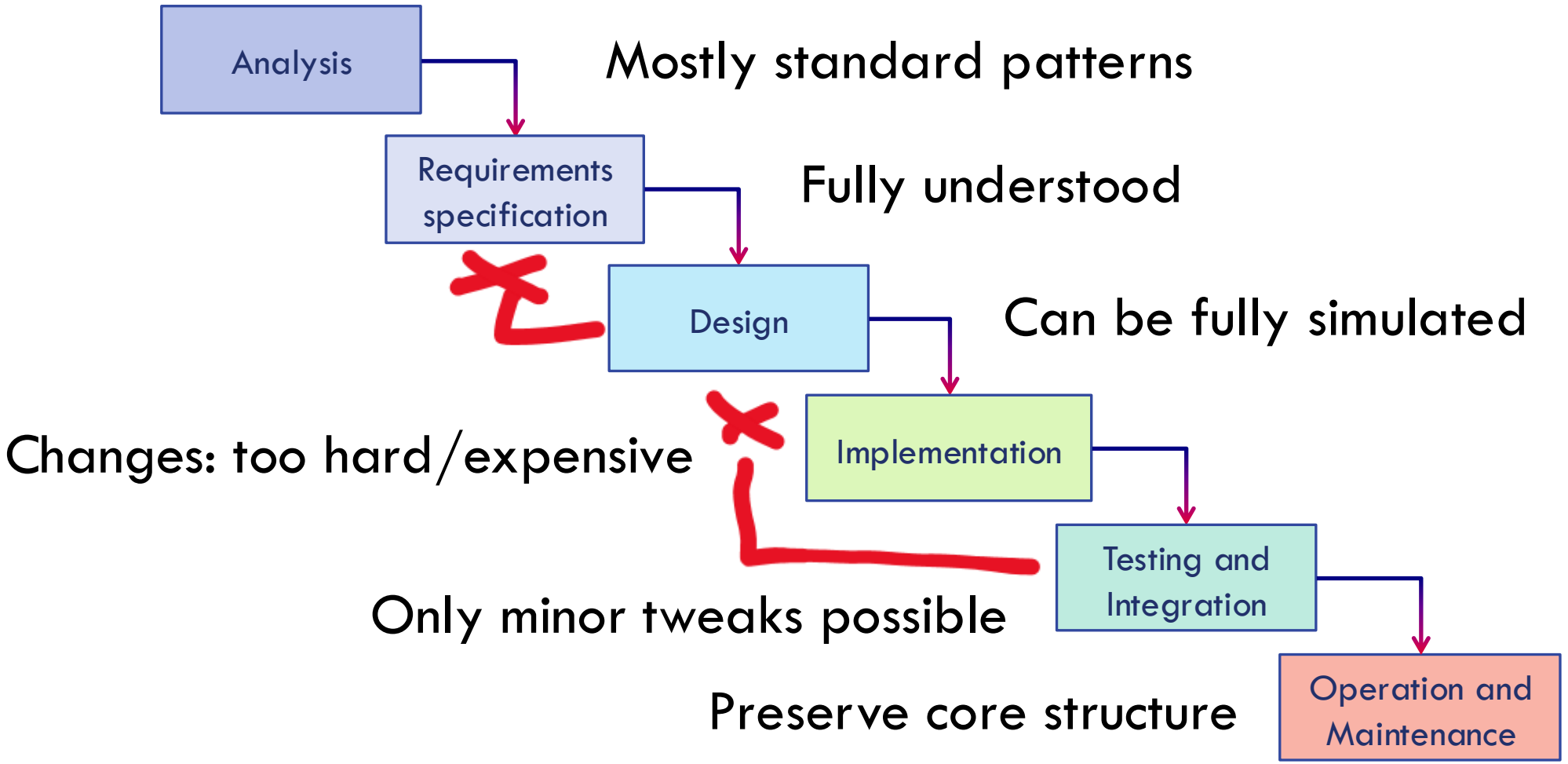


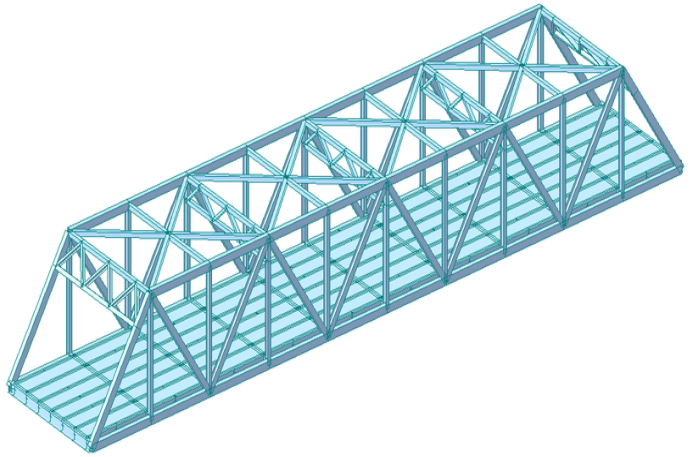


Figure 48 Bridge immediately prior to the collapse, March 15, 2018



Figure 49 Bridge immediately after the collapse, March 15, 2018 – camera #2
Employees engaged in re-tensioning the PT bars in diagonal 11

Waterfall Method: FIU Bridge





Analysis

Requirements specification

Design

Implement

Testing and Integration

Operation and Maintenance

"Must have redundancy"

PC Speak for: use traditional design

FIU: "Must be unique"

=> Single truss design: first/only in the world

Primary engineer: wrong calculations

Second reader: sleeping on job

As per faulty design

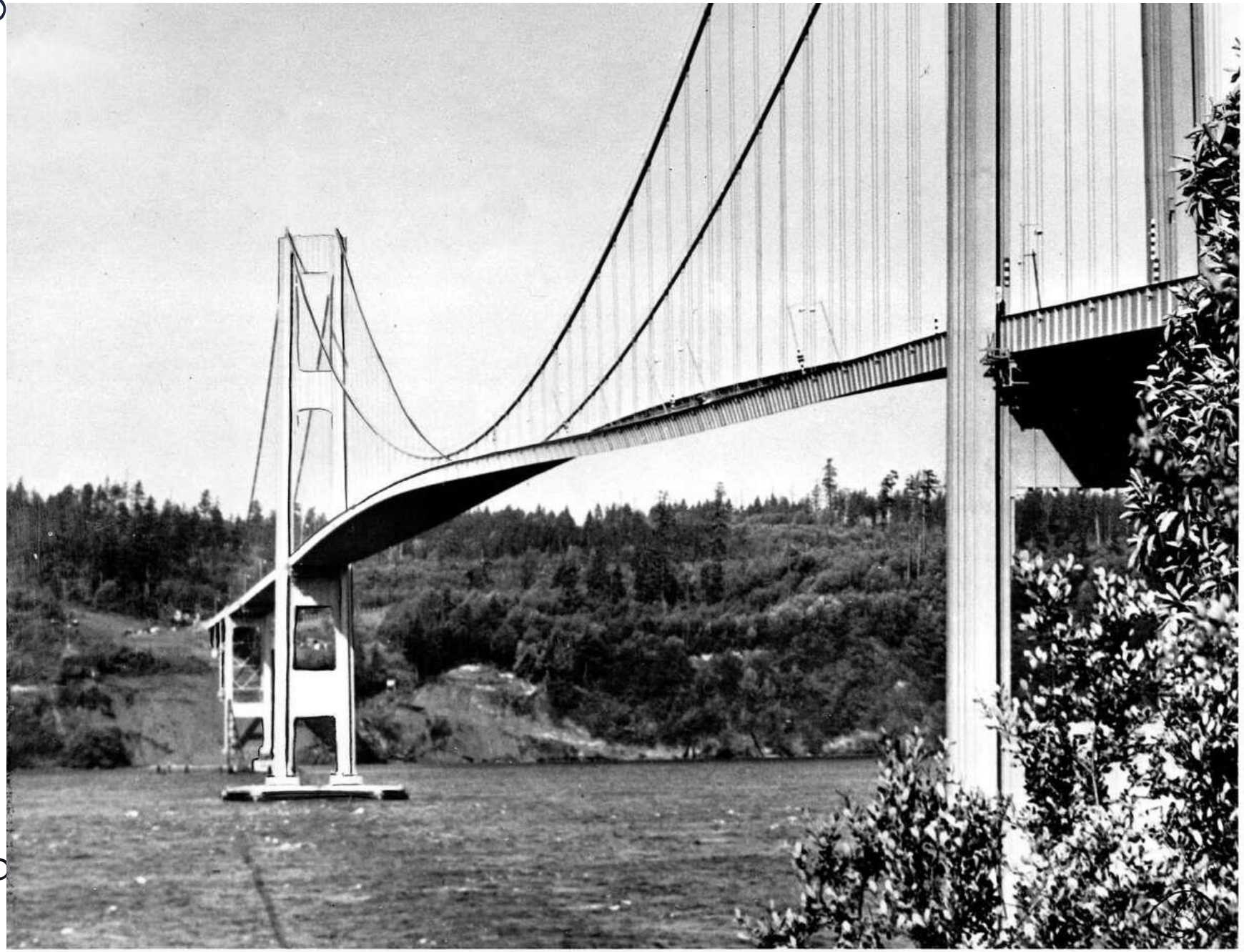
Cracks, fix: loosen tensioner + re-tension

after loosening: bridge collapses

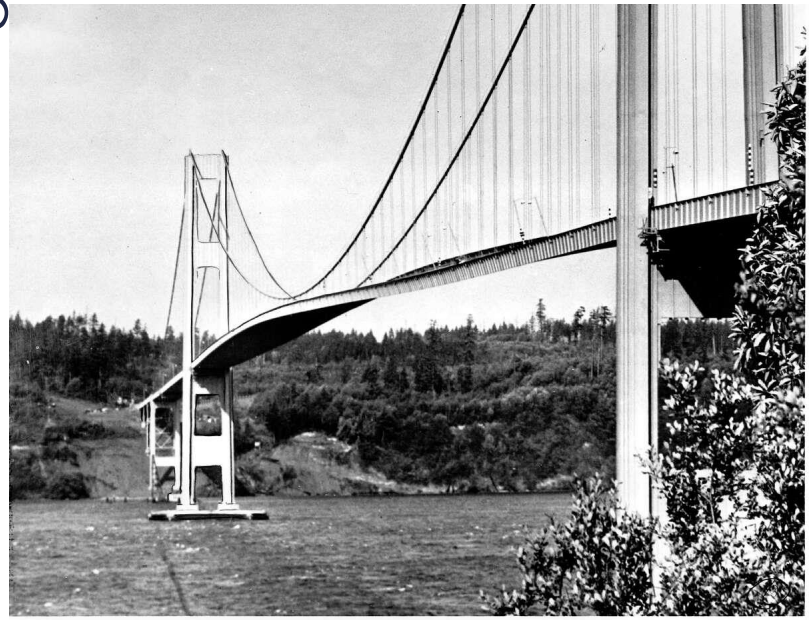
Why pure Waterfall cannot work for software

- Preceding phases must be perfect
- **Soft**ware, infinitely changeable
- Pure model does not allow 'go around' *(for building makes sense)*
- Coding problems may require redesign
  - Testing issues may require redesign
  - User acceptance issues may require …
  - Operational issues may …
  - Maintenance issues …
- Big projects fail much more often than small projects

“Wicked” problems



“Wicked” problems





Tacoma Narrows bridge 7 November 1940

- Unknown unknowns
  - Cannot be anticipated
  - Only visible late:
    - in implementation/testing/operation
- Software is full of wicked problems
- Software is much more complex than hardware

Problem matrix

Wicked problem = Unknown unknown

can't be foreseen, is not solved

Foreseen → **Solved ↓**	**Yes, clear**	**No, foggy**
Yes	*Known known* Use std soln	*Unknown known* refactor
Not yet	*Known unknown* Fix in req./analysis	*Unknown unknown* Go back to start
Show up at:	Req./analysis	Coding/testing/ deployment

Change and the Waterfall Method

- The cost of change increases exponentially with time
  - conservative design decisions motivated by fear of change
    - a change late in the process costs 1000 times as much as a change early in the process
      - five minutes to write a spec
      - two days to program the feature
      - two weeks to test it before deployment
      - month to write a patch that fixes a problem after deployment



CSC3003S ASD02
Software engineering methods

A. Faster, Better, Cheaper (Pick Two)

B. Problems

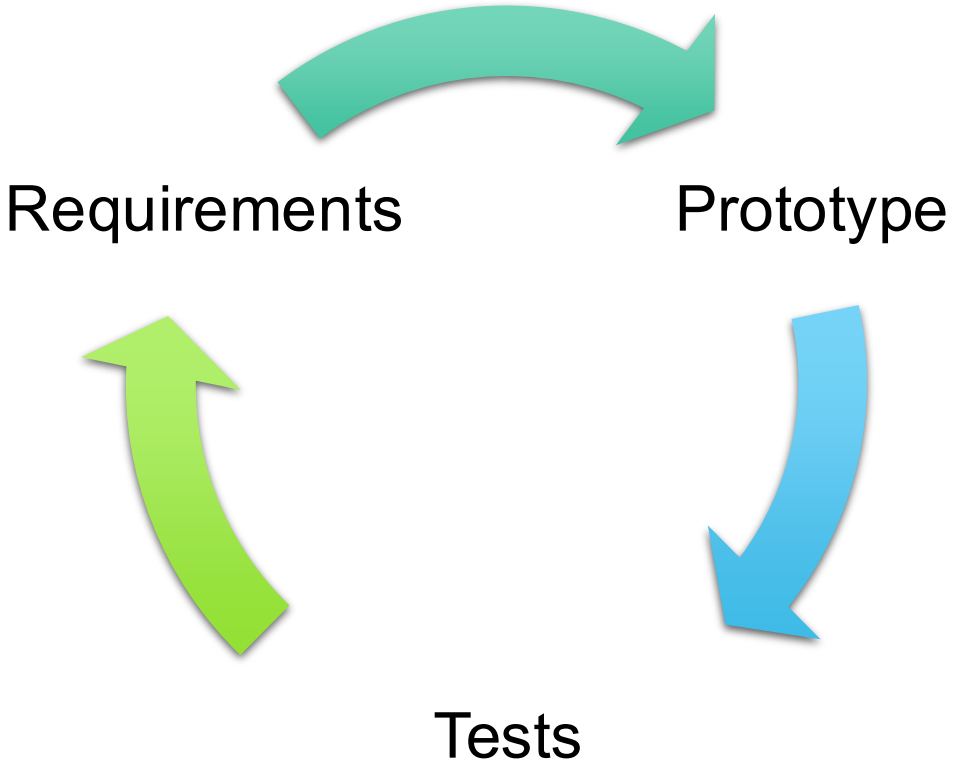
C. Naïve Approaches

D. Iteration

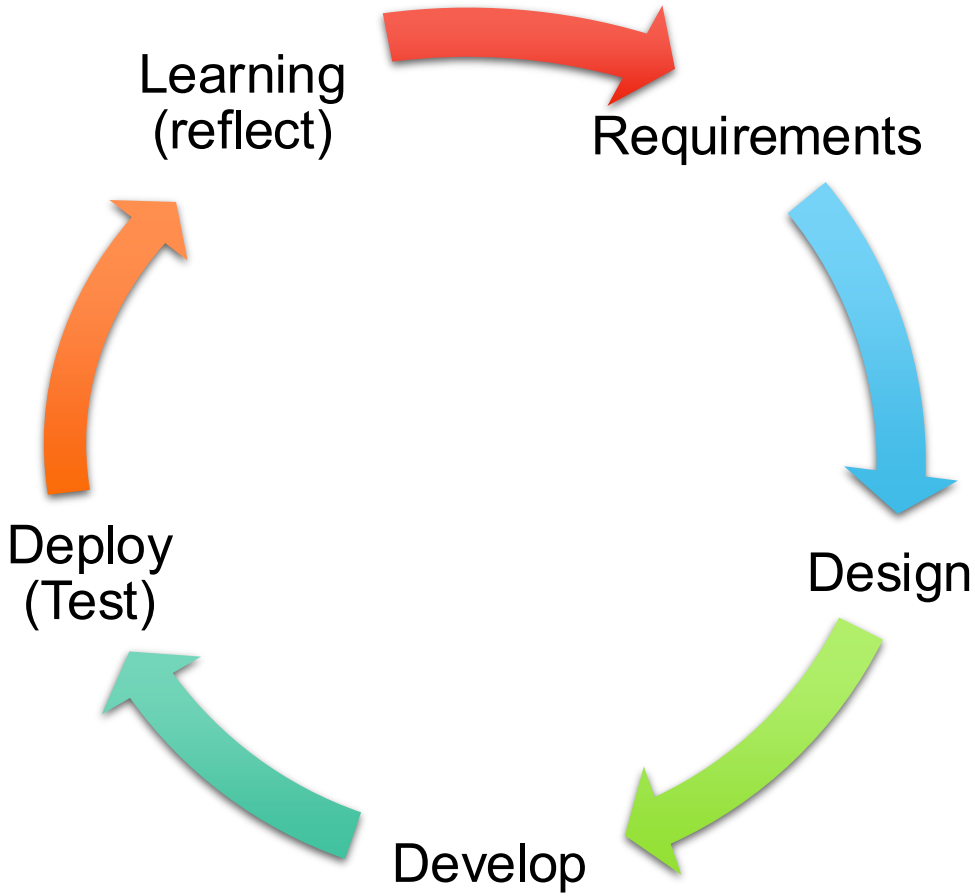
More modern processes

- More lightweight than waterfall
  - **Less documentation**
  - **Fewer procedures**
- **Release multiple versions during development**
  - Parallel development
  - Prototypes
- *Skateboard: Only do what is required*
  - No adding in extra requirements
- Design for change
  - Change is inevitable

Alternative Ideas: Prototyping



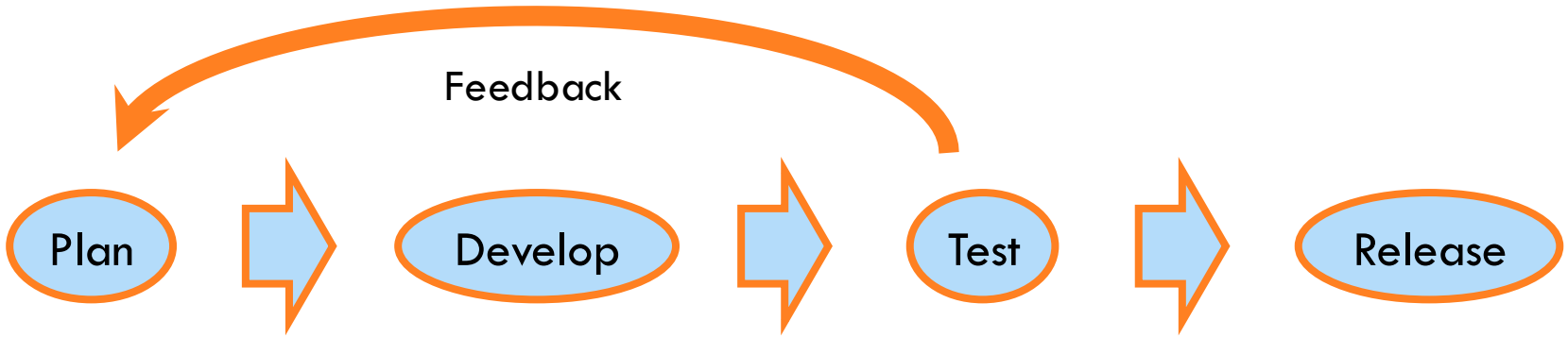
Rapid Application Development



Iterative (aka incremental) process

- Plan development
- Undertake development
- Generate a prototype
- Get user feedback
- Develop again

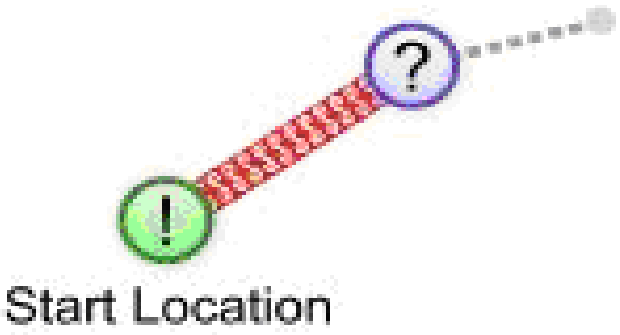
Iterative software development: a process that reaches the goal in a series of ever improving delivery cycles



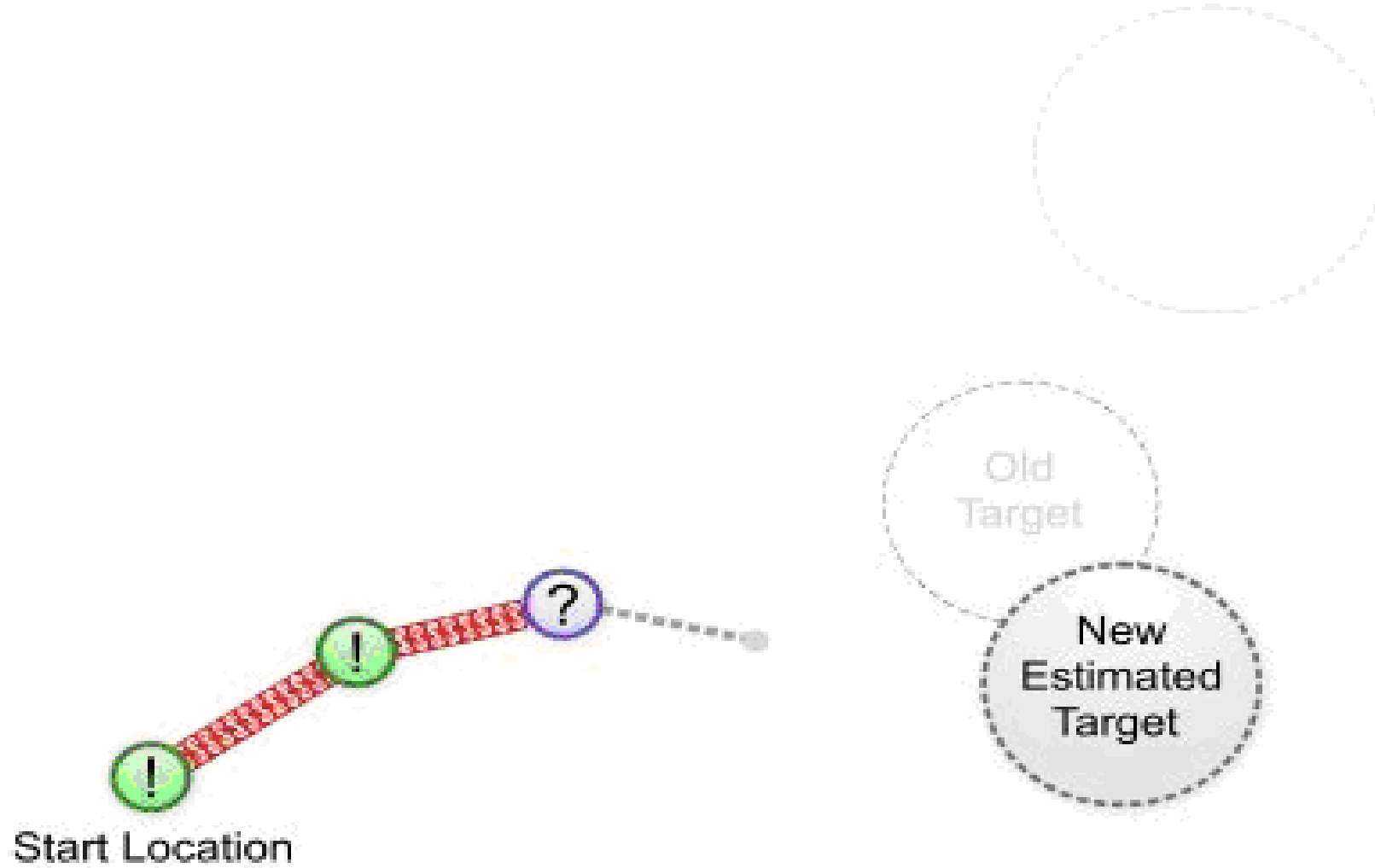
Iterative process iteration 1



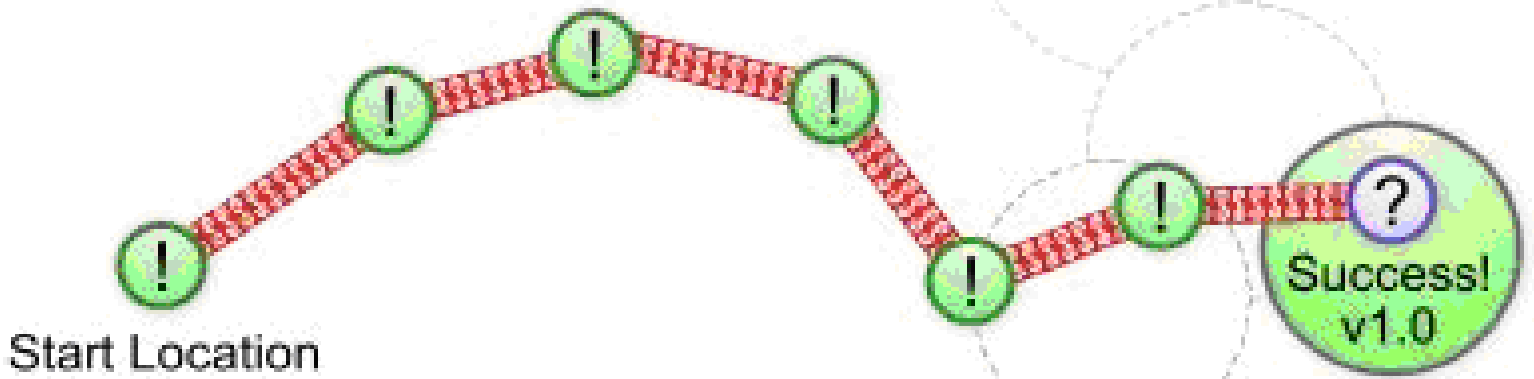




Iterative process iteration 2



Iterative process success



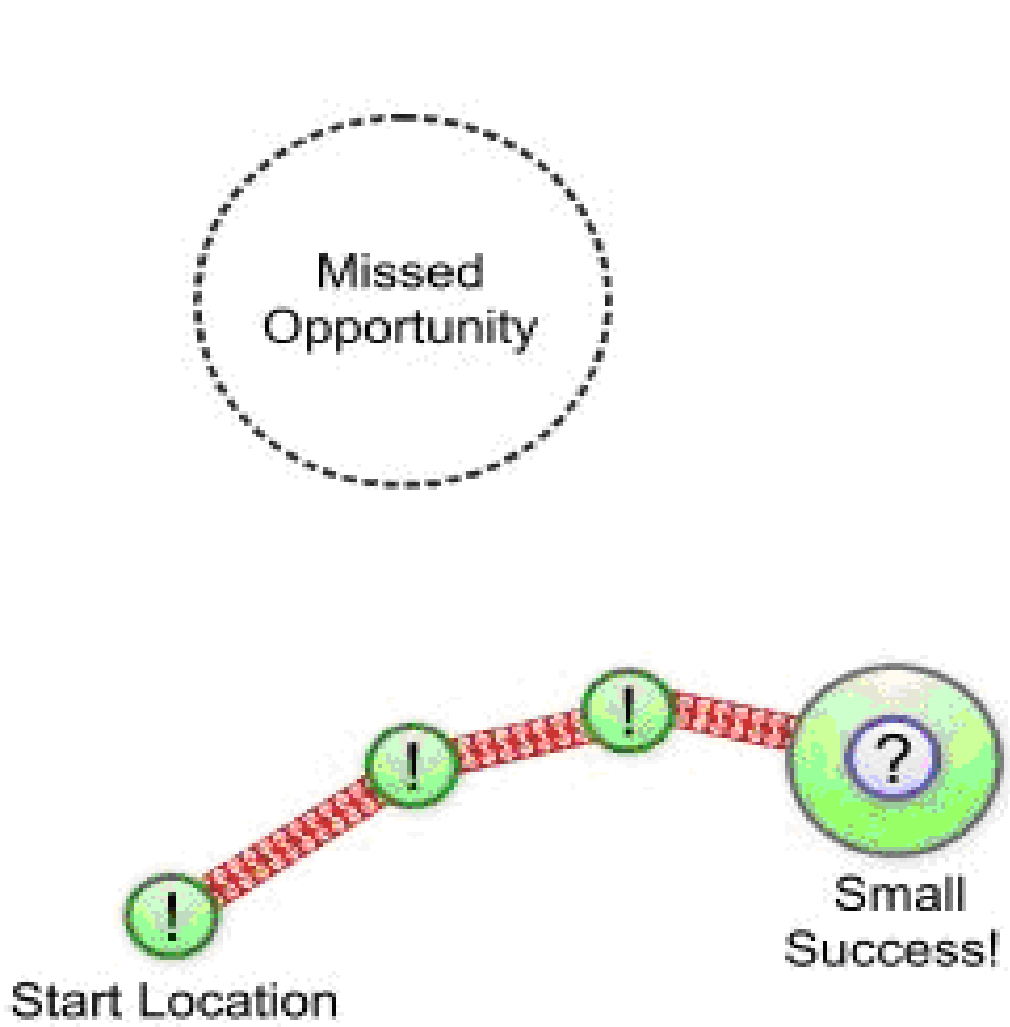
Iterative process – the big difference

- Instead of 18 months to create and evaluate one big project
  → Build and show a new version to users every 2 to 4 weeks.
- Requires
  - **team members close** together and **close to customer.**
  - team members agree on good ideas in hours, not months.
  - teams become experts through
    intense hands-on problem solving and testing.
- Ends up with real systems meeting the users' real, tested needs,
  not "perceived" needs.

Iterative process – the benefits

- Favoured by small start up companies
  - greatly reduce risk of a project failing.
    - only one shot at success, steer your way to success using information instead of launching blindly into the unknown.
  - Long term: iterative development delivers more value sooner, with lower overall risk
- Project is intensely focused
  - in only working on high priority features, many alternative concepts never get explored.
    - good ideas can be lost

Iterative process – the danger



Examples of iterative software development methods

Some important flavours with a few key features:

- Agile ("Agile Manifesto")
  - Mini software projects; face-to-face communication
- RAD: Rapid Application Development (James Martin - *uncle Bob*)
  - Voice of customer; Collaborative; rigid schedule
- XP: Extreme Programming (Kent Beck)
  - Design on the fly
  - unit testing of all code
  - pair programming
  - refactoring
- Scrum (Takeuchi, Nonaka, Schwaber)
  - Teams scrum down in short iterations (sprints)
    - empirical process